

PDB data structure:

```
from Bio.PDB import PDBParser
parser = PDBParser(QUIET=True)
structure = parser.get_structure("protein", "example.pdb")
```

Structure → Model → Chain → Residue → Atom
Protein → Chains → Residues (Amino Acids) → Atoms

Level	Class	Represents	Access Example
Structure	<code>Bio.PDB.Structure.Structure</code>	Whole macromolecule (usually one PDB file)	<code>structure</code>
Model	<code>Bio.PDB.Model.Model</code>	One 3D model (for NMR structures, there can be several)	<code>structure[0]</code>
Chain	<code>Bio.PDB.Chain.Chain</code>	A single polypeptide chain (A, B, C, etc.)	<code>structure[0]["A"]</code>
Residue	<code>Bio.PDB.Residue.Residue</code>	A single amino acid or ligand	<code>structure[0]["A"][(' ', 50, ' ')]</code>
Atom	<code>Bio.PDB.Atom.Atom</code>	A specific atom within a residue	<code>structure[0]["A"][(' ', 50, ' ')]['CA']</code>

Model

- Some PDB files (especially from NMR experiments) have multiple “models” — each one is a possible 3D conformation of the same protein.
- In X-ray or cryo-EM data, usually only one model is present.
- Think of a **model** as a snapshot or frame.

Chain

- A protein can have multiple **chains** — for instance, hemoglobin has four (two alpha and two beta).
- Each chain represents one **continuous polypeptide** (one linear sequence of amino acids).
- In PDBs, chains are labelled by a single letter: A, B, C, etc.

Residue (Amino Acid)

- Each **residue** corresponds to one amino acid in the sequence.

- It's identified by:
 - **Residue name:** 3-letter code (e.g., ALA, LYS, TYR)
 - **Residue sequence number:** e.g., 23, 24, 25 (can have insertion codes like 23A)
 - **Chain ID:** to know which chain it belongs to

In a PDB line you might see:

```
ATOM    123  CA  LYS A   23  ...
```

This means:

Atom #123 is the **alpha carbon (CA)** of the **lysine residue (LYS)** at position 23 in **chain A**.

Atom

- This is the most detailed level. Every atom has:
 - Atom name (e.g. CA, CB, N, O)
 - 3D coordinates (x, y, z)
 - Element type (C, N, O, S, etc.)
- Example line:

```
ATOM      1  N   MET A      1      11.104  13.207   5.312   1.00 20.00           N
```

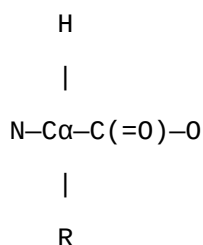
So the PDB file ultimately is a **list of atoms** with coordinates and labels describing which residue and chain they belong to.

Structural Level	Biological Meaning	Example
Primary	Linear sequence of amino acids	MKTFFVFTTQRYDEQEELFFQN
Secondary	Local structures like α -helices and β -sheets	Helix formed by residues 10–20
Tertiary	Full 3D folding of one chain	Entire chain A's 3D shape
Quaternary	Assembly of multiple chains	Chains A and B forming a complex

Biology understanding:

What is the alpha carbon (C α)?

Every amino acid (except glycine, which is slightly special) has the same basic backbone structure:



- **N** → nitrogen atom (part of the amino group)
- **C α** → *alpha carbon* — the central carbon that connects to:
 - The amino group (N)
 - The carboxyl group (C=O)
 - A hydrogen atom (H)
 - A unique **side chain (R)** — this is what makes each amino acid different

```
file models chains residues atoms ca_atoms      exp_method \
0 4WXT.pdb    1    1    108  829    111 X-RAY DIFFRACTION
```

Column	Meaning	Interpretation for 4WXT . pdb
file	The name of the PDB file you analysed. Each file represents one experimentally determined protein structure (sometimes a protein complex).	4WXT . pdb is the identifier for this specific protein's structure.
models	The number of <i>structural models</i> in the file. Some experimental methods (like NMR) include multiple conformations — each “MODEL” is one 3D snapshot.	1 means this file has a single model — typical for an X-ray or cryo-EM structure.
chains	The number of <i>polypeptide chains</i> (continuous amino acid sequences) in the structure. A protein complex (e.g., hemoglobin) may have several chains (A, B, C, etc.).	1 means it's a single-chain protein — not a multi-subunit complex.
residues	The number of <i>standard amino acid residues</i> in that chain (after filtering out waters, ligands, etc.). This is the sequence length represented in the structure.	108 amino acids in total — roughly a small, compact protein domain.
atoms	The total number of <i>individual atoms</i> within those residues (all backbone and side-chain	829 total atoms, consistent with a small protein of 108 residues.

Column	Meaning	Interpretation for 4WXT . pdb
	atoms). Each residue typically has 7–15 atoms, depending on the side chain.	
ca_atoms	The count of <i>alpha carbon (Cα)</i> atoms — one per residue if the structure is complete. Cα atoms form the backbone “spine” of the protein and are used to compute residue–residue distances.	111 Cα atoms — roughly one per residue, plus perhaps 2–3 extra due to alternate conformations or chain numbering quirks.

So in short, this record tells you that **4WXT** is a **single-chain protein** of about **108 residues**, represented by roughly **829 atoms**, and that the structure was obtained by **X-ray diffraction**.

The slight mismatch between **residues** (108) and **ca_atoms** (111) is normal — it often reflects a few insertion-coded residues (like 23A, 23B) or alternate conformations resolved in the crystal.

What a chain is

- A **chain** is one **continuous polypeptide** (a single linear string of amino acids linked by peptide bonds) as it exists in the 3D structure.
- In a PDB, each chain gets a **chain ID** (A, B, C, ...) so atoms/residues can be grouped.
- Biologically, one chain typically corresponds to the translation product of **one gene (one polypeptide)**, though some proteins are cleaved after translation (see below).

Where chains sit in the hierarchy

- **Atom** → belongs to a **Residue** (amino acid) → belongs to a **Chain** → belongs to a **Structure/Model**.
- Your ML contact map (for this assignment) is usually **within a single chain** (intra-chain/tertiary structure), unless you explicitly model inter-chain interfaces (quaternary structure).

Processing needed:

High level overview:

residue–residue contact prediction, you'll:

1. Parse the PDB file into chains
2. Per chain extract each residue's **C α atom coordinates** (the alpha carbon is the central carbon of every amino acid backbone).
3. Compute **pairwise distances** between all residues.
4. Define a **contact** if distance < 8 Å.

So your processing steps look like:

PDB file → Per chain → parse atoms → group by residue → extract C α → compute distance matrix → build contact map

Possible feature engineering re “b_factor”

Finding Cas:

processing pipeline (concepts first)

1. Choose the model

- Some PDBs (e.g., NMR) contain multiple MODEL/ENDMDL blocks. Either:
 - take the **first model** for simplicity, or
 - loop over models and average or pick one by some rule.
- If there's no MODEL record, there's just one model.

2. Iterate by chain

- Each chain is a polypeptide (A, B, C...).
- You'll usually build a C α list **per chain** because contact maps are chain-specific unless you want inter-chain contacts.

3. Iterate by residue within a chain

- Each residue is identified by (resname, resseq, icode):
 - resname: 3-letter code (e.g., ALA, LYS)
 - resseq: residue number (can be non-contiguous)
 - icode: insertion code (e.g., 23A)

- Ignore **waters** (HOH) and **hetero residues** unless you intentionally want them (we usually don't for protein contact maps).

4. Pick the C α atom for that residue

- Look for an atom with **name CA** inside the residue's atoms.
- Handle **alternate locations** (altlocs, e.g., A, B conformers):
 - Prefer the altloc with the **highest occupancy** (or altloc A if you want a simple rule).
- If **C α is missing** (happens in disordered regions), skip that residue or mark as missing.

5. Store one C α per residue

- Keep: (chain_id, residue_index_in_chain, pdb_resseq_info, xyz) where xyz is a float3.
- Note: PDB numbering can have gaps and insertions. For ML, you'll often reindex residues **0..L-1** per chain.

6. (Optional but important) Map PDB residues to your input sequence

- If your model's input is a **query sequence**, you must map **PDB residues** $\rightarrow$ **sequence positions**.
- Build the **observed sequence from ATOM records** (standard residues only), then run a **global alignment** (e.g., Biopython's pairwise2 or parasail) between:
 - PDB-derived sequence
 - your query sequence
- Use the alignment to create a mapping **pdb_index** $\rightarrow$ **sequence_index**. This lets you place C α distances correctly into an $L \times L$ matrix aligned with your input sequence.

Processing notes:

Each residue has one alpha carbon with 3d co-ordinate x,y,z

Protein of length L will have L x 3 matrix of co-ordinates of all alpha-carbons

We can define an LxL distance matrix as follows:

$$D_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2}$$

If you have L residues, this gives you an $L \times L$ matrix of pairwise distances.

- The matrix is **symmetric** ($D[i, j] = D[j, i]$) and has zeros along the diagonal ($D[i, i] = 0$).

convert D_{ij} into binary contact map by thresholding

$$D[i, j] < 8 \text{ \AA} \rightarrow 1 \text{ else } 0$$

Incorporating struct data:

Incorporate **structural data from similar sequences** (the assignment's twist)

You'll boost performance by injecting **template information** from homologs with known structures:

Pipeline:

1. **Find similar sequences** to your query (e.g., with **MMseqs2** or **BLAST** against your provided PDB set).
2. For each hit with a structure:
 - **Align** query $\leftrightarrow$ template (global or local, keep gaps).
 - **Project** the template's **C α distance map** into query coordinates via the alignment:
 - If query $i \leftrightarrow$ template i' and query $j \leftrightarrow$ template j' (both non-gap), set $D_{\text{template}}[i, j] = D_{\text{template_struct}}[i', j']$, else mark as missing.
 - Convert to features: e.g., **binned distances** (one-hot over bins like $<6\text{\AA}$, $6-8\text{\AA}$, $8-12\text{\AA}$, $>12\text{\AA}$) or a simple **template contact prior** $C_{\text{template}} = 1[D < 8\text{\AA}]$.
 - **Weight** by sequence identity/coverage (down-weight distant homologs) and **aggregate** across multiple templates (max, mean, attention-weighted).
3. **Fuse** template features with ESM2 pairwise features:
 - **Concatenate**: $\text{pair_fused} = \text{concat}(\text{pair}, \text{template_bins}, \text{template_contact_prior}, \text{identity_weight})$
 - Or **gate** them: $\text{pair_fused} = \text{pair} + \sigma(W * \text{template}) \odot U * \text{pair}$
 - Or add as a **bias** to logits: $\text{logits} = \text{head}(\text{pair}) + w * \text{template_prior}$